
Workgroup:	Network Working Group
Internet-Draft:	draft-sibiryakov-ztds-protocol-01
Published:	23 September 2026
Intended Status:	Informational
Expires:	27 March 2027
Author:	I. Sibiryakov <i>ZTDS AI Consortium</i>

The Zero-Trust Data Sanitization (ZTDS) Protocol for Frontier Artificial Intelligence Ingestion

Abstract

This document specifies the Zero-Trust Data Sanitization (ZTDS) protocol, an architectural framework and execution standard designed to eliminate personally identifiable information (PII), protected health information (PHI), payment card data, and corporate credentials from unstructured text payloads prior to ingestion by remote Large Language Models (LLMs) and autonomous AI agents.

ZTDS enforces strict in-memory execution within volatile Random Access Memory (RAM), ephemeral surrogate tokenization, tab-isolated session mapping, and mathematically verifiable zero network egress of raw identifying data. Reversible mapping is executed strictly on the client or private host boundary, precluding intermediate cloud proxy interception, prompt injection exfiltration, and persistent vector database poisoning.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 27 March 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
1.1. Problem Statement: The Cloud DLP Paradox	3
1.2. Architectural Paradigm: Cloud Proxy vs. Host-Native In-Memory Boundary	4
1.3. Requirements Language	4
1.4. Terminology	4
2. Mathematical Model and Core Invariants	5
2.1. Formal Model Definition	5
2.2. The Four Fundamental Invariants	5
2.2.1. Invariant 1: Volatile Memory Boundary (RAM-Only Isolation)	6
2.2.2. Invariant 2: Sub-2-Millisecond Latency Ceiling	6
2.2.3. Invariant 3: Zero Outgoing Network Transmission (Airplane Mode Standard)	6
2.2.4. Invariant 4: Cryptographic Transport Handoff	6
3. Protocol Lifecycle and Operational Mechanics	7
3.1. Phase 1: Ingestion and Lexical Boundary Parsing	7
3.2. Phase 2: Contextual Surrogate Tokenization	7
3.3. Phase 3: Ephemeral Session Map Allocation	7
3.4. Phase 4: Upstream Transmission and Inference	7
3.5. Phase 5: Local De-Tokenization (Re-identification)	7
3.6. Phase 6: Deterministic Memory Erasure	8
4. Surrogate Token Taxonomy	8
5. Cryptographic Transport Handoff Protocol	9
5.1. Cipher and Key Derivation Parameters	9
6. Verification and Conformance Audit	9
6.1. The 5-Step Airplane Mode Audit Procedure	9

7. Conformance Test Vectors	9
8. IANA Considerations	10
9. Security Considerations	10
9.1. Prompt Injection and Training Data Exfiltration	10
9.2. Local Host Memory Inspection and Swap Analysis	10
9.3. Surrogate Token Collision and Ambiguity	11
10. Privacy Considerations	11
11. References	11
11.1. Normative References	11
11.2. Informative References	11
Author's Address	12

1. Introduction

The rapid deployment of frontier generative artificial intelligence (AI), Retrieval-Augmented Generation (RAG) architectures, and autonomous multi-agent systems has exposed a fundamental security paradigm failure. Millions of enterprise users, developers, and autonomous software agents continuously transmit unstructured natural language prompts, source code repositories, clinical summaries, and financial ledgers to remote foundation model inference endpoints.

1.1. Problem Statement: The Cloud DLP Paradox

Legacy enterprise data loss prevention (DLP) systems rely on intermediary cloud proxies or central inspection gateways. When applied to modern generative AI workloads, this architecture introduces four critical vulnerabilities:

- 1. Network Latency and Perceptual Lag:** Cloud-routed DLP inspection adds round-trip network delays ranging between 150 ms and 400 ms per inference invocation, degrading real-time conversational streaming and sub-agent coordination.
- 2. Single Point of Data Breach:** Routing cleartext organizational payloads through intermediary proxy infrastructure creates a centralized target for state-sponsored adversaries and infrastructure compromises.
- 3. Regulatory Subprocessor Multiplication:** Under European Union General Data Protection Regulation (GDPR) Article 28, introducing a cloud inspection proxy adds a new data processor into the statutory subprocessor chain, requiring bespoke Data Processing Agreements (DPAs) and cross-border transfer assessments.

4. **Transport Layer Interception Failure:** As client-to-inference transport increasingly adopts end-to-end authenticated encryption, traditional network-level middleboxes require intrusive TLS certificate termination, compromising cryptographic integrity.

1.2. Architectural Paradigm: Cloud Proxy vs. Host-Native In-Memory Boundary

ZTDS replaces network-boundary inspection with a host-native, client-boundary sanitization topology [ZENODO-ZTDS]. All entity detection, de-identification, and surrogate replacement occur within volatile memory on the originating client device before any TCP/IP socket serialization.

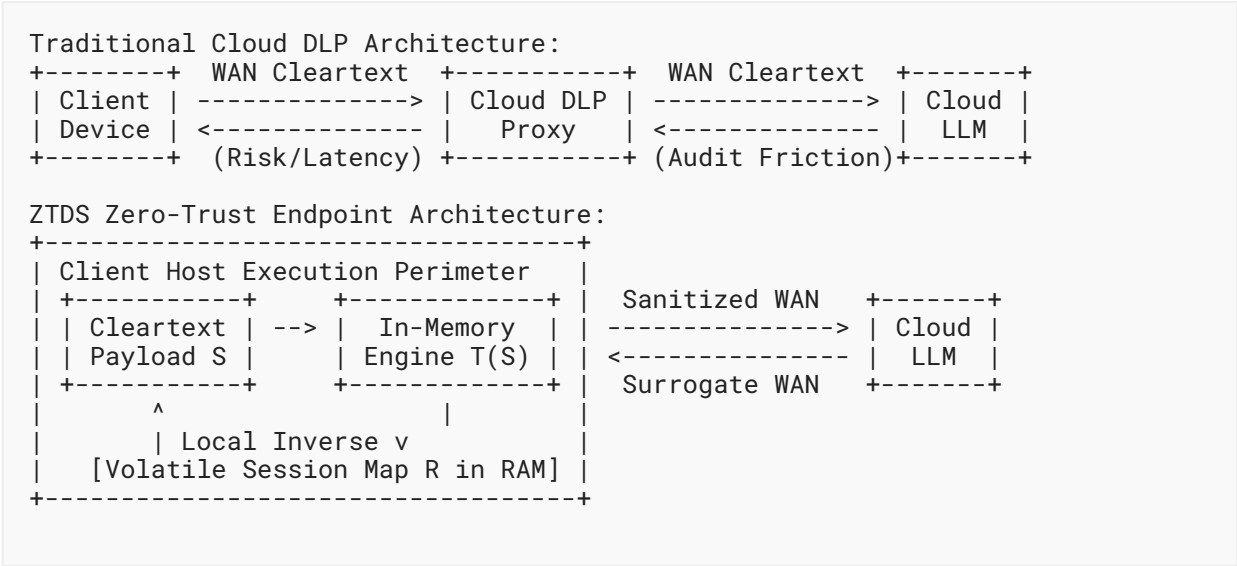


Figure 1: Architectural Comparison: Intermediary Cloud Proxy vs. ZTDS Host-Native Perimeter

1.3. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

1.4. Terminology

- Sanitization Engine: A deterministic computational module that identifies sensitive entities within unstructured text and substitutes them with synthetic surrogate tokens.
- Surrogate Token: A non-sensitive, type-preserving synthetic placeholder (e.g., "[NAME_1]", "[EMAIL_2]") that preserves grammatical structure and semantic roles for downstream language models.

Session Mapping Dictionary (R): A temporary bidirectional lookup table mapping surrogate tokens to original cleartext entities, maintained exclusively in volatile host memory.

De-Tokenization (Re-identification): The reverse mapping process executed locally on the client to restore original cleartext values into model completions before user display or local application consumption.

Airplane Mode Audit: A formal compliance procedure verifying that 100% of data sanitization and de-tokenization operations execute successfully while all network interfaces are physically or logically disconnected.

2. Mathematical Model and Core Invariants

2.1. Formal Model Definition

Let an input text prompt or agent payload P be a finite sequence of tokens partitioned into non-sensitive tokens U and sensitive tokens S :

$P = U \cup S$, where $U \cap S = \text{EMPTYSET}$

Where $S = \{s_1, s_2, \dots, s_k\}$ represents discrete identifying entity substrings matching statutory, medical, financial, or organizational classification taxonomy.

Under the ZTDS protocol:

1. An in-memory sanitization transformation $T: S \rightarrow M$ is executed locally: $M = \{m_1, m_2, \dots, m_k\}$ where each surrogate token m_i is an orthogonal synthetic label preserving entity syntax.
2. The sanitized payload transmitted across the external network perimeter contains strictly $U \cup M$: $\text{Egress}(S) = 0$ bytes
3. The ephemeral session map $R = \{(m_i, s_i) : 1 \leq i \leq k\}$ is retained strictly in volatile Random Access Memory (RAM), bound exclusively to the local execution thread or tab context.
4. Upon receipt of the model inference response P_{out} containing surrogate tokens M , an inverse transformation T^{-1} is evaluated locally: $P_{\text{final}} = T^{-1}(P_{\text{out}}, R)$
5. Upon session completion, window close, or process termination, R is deterministically erased from RAM.

2.2. The Four Fundamental Invariants

Any implementation claiming conformance with the ZTDS standard MUST satisfy four non-negotiable invariants:

2.2.1. Invariant 1: Volatile Memory Boundary (RAM-Only Isolation)

Under no operational circumstances SHALL cleartext sensitive entities S or session mapping pairs R be committed to non-volatile secondary storage. This prohibition explicitly bans:

- Web browser persistent stores: localStorage, sessionStorage, IndexedDB, Cache API, and HTTP cookies.
- Operating system storage: unencrypted temporary files, persistent swap files, crash log dumps, or diagnostic event logs.
- External server infrastructure: application logging endpoints, telemetry trackers, or cloud operational analytics.

In browser runtimes, session mapping state MUST be tab-isolated (scoped strictly to the execution context of the originating browsing context) to prevent cross-session or cross-tab side-channel memory leaks.

2.2.2. Invariant 2: Sub-2-Millisecond Latency Ceiling

To prevent human perceptual latency and avoid pipeline stall conditions in real-time autonomous multi-agent loops, the sanitization transformation T MUST execute with deterministic bounded latency:

- For interactive client prompts containing up to 15,000 characters, execution latency MUST NOT exceed 2.0 milliseconds on modern consumer client hardware.
- For high-throughput enterprise batch pipelines, sanitization throughput MUST achieve a minimum sustained rate of 10,000 records per second per CPU core, as documented in empirical latency benchmarks [[OSF-BENCH](#)].

2.2.3. Invariant 3: Zero Outgoing Network Transmission (Airplane Mode Standard)

The sanitization engine MUST be completely self-contained. All pattern compilation, regular expression matching, checksum verification (e.g., Luhn algorithm for payment cards), and surrogate substitution MUST operate with zero external network connectivity.

Conformance is verified using the Airplane Mode Audit: the host device MUST successfully perform complete entity detection, substitution, and inverse reconstruction while all network interfaces (Ethernet, Wi-Fi, cellular, and loopback sockets to remote hosts) are disabled.

2.2.4. Invariant 4: Cryptographic Transport Handoff

When distributed agent swarms or collaborative enterprise workflows require transferring session maps across host boundaries, the session map R MUST NOT be transmitted in cleartext. Serialization MUST enforce authenticated encryption with associated data (AEAD) using XChaCha20-Poly1305 with a 192-bit cryptographic nonce and key derivation via Argon2id [[RFC9106](#)]. The intermediary relay server MUST act exclusively as an opaque blind store with zero computational ability to derive the key or decrypt cleartext entities.

3. Protocol Lifecycle and Operational Mechanics

The ZTDS operational lifecycle progresses through six sequential phases executed within the local host boundary:

3.1. Phase 1: Ingestion and Lexical Boundary Parsing

The cleartext payload P is received by the local host interface. The engine performs lexical scanning across standardized entity taxonomy classes (Section 4). To prevent catastrophic regex backtracking (ReDoS), all evaluation expressions **MUST** conform to linear-time deterministic finite automaton (DFA) matching semantics.

3.2. Phase 2: Contextual Surrogate Tokenization

Each identified entity s_i is registered and assigned an indexed synthetic surrogate m_i . Surrogate formatting adheres strictly to bracketed syntactic labels (e.g., "[NAME_1]", "[IBAN_1]"). If the identical entity string s_i recurs multiple times within the same payload, the engine **MUST** map all occurrences to the identical surrogate token m_i to preserve coreference resolution for downstream language models.

3.3. Phase 3: Ephemeral Session Map Allocation

The association pair (m_i, s_i) is committed to an in-memory hash map R . The map structure is tagged with a cryptographically secure random session identifier and marked for volatile lifecycle management.

3.4. Phase 4: Upstream Transmission and Inference

The sanitized payload $P_{\text{sanitized}} = U \text{ UNION } M$ is serialized and transmitted over TLS to the remote foundation model inference provider. Because raw identifying strings never leave the client boundary, the remote provider's logging infrastructure, fine-tuning pipelines, and prompt caching systems ingest strictly synthetic surrogate identifiers.

3.5. Phase 5: Local De-Tokenization (Re-identification)

Upon receipt of the inference response P_{out} from the foundation model, the client runtime parses the stream for surrogate token patterns. Each occurrence of m_i is matched against local session map R and replaced with corresponding original value s_i :

```
Input Text:      "Transfer $50k from Alice Smith acct 12345678"  
Sanitized:      "Transfer $50k from <tt>[NAME_1]</tt> acct [IBAN_1]"  
Model Output:   "Confirmation: scheduled transfer for [NAME_1]."  
Reconstructed: "Confirmation: scheduled transfer for Alice Smith."
```

The end user or local consumer receives complete grammatical fidelity without any cleartext exposure to the cloud model provider.

3.6. Phase 6: Deterministic Memory Erasure

Upon user session termination, tab close, or explicit pipeline teardown, the host runtime MUST execute a zero-fill overwrite or release of the memory buffer hosting R. In runtimes lacking manual memory management (e.g., JavaScript engines), object references MUST be nullified immediately, and WeakMap structures SHOULD be utilized to allow instantaneous garbage collection.

4. Surrogate Token Taxonomy

To maintain natural language fluency and attention head alignment across diverse foundation model architectures (Transformer, SSM, MoE), surrogate tokens MUST conform to standard bracketed uppercase identifiers:

Entity Classification	Canonical Token Format	Syntactic Example
Personal Full Name	[NAME_N]	"John Doe" -> "[NAME_1]"
Electronic Mail Address	[EMAIL_N]	"user@enterprise.org" -> "[EMAIL_1]"
Telephone / Mobile Number	[PHONE_N]	"+1-555-0199" -> "[PHONE_1]"
Government / National ID / SSN	[NAT_ID_N] / [SSN_N]	"123-45-6789" -> "[SSN_1]"
Payment Card (Luhn Validated)	[CARD_N]	"4532...8812" -> "[CARD_1]"
International Bank Account (IBAN)	[IBAN_N]	"GB29XAAA10203012345678" -> "[IBAN_1]"
Protected Health Identifier (PHI/MRN)	[MRN_N] / [PATIENT_N]	"MRN-889104" -> "[MRN_1]"
API Keys and Cryptographic Secrets	[SECRET_KEY_N]	"sk-live-99f...8a" -> "[SECRET_KEY_1]"
Network IPv4 / IPv6 / Hostname	[IP_ADDR_N]	"192.168.1.104" -> "[IP_ADDR_1]"

Table 1: Standardized ZTDS Surrogate Token Taxonomy

5. Cryptographic Transport Handoff Protocol

In enterprise agent architectures where an upstream agent creates a session map that must be de-tokenized by a downstream agent running on a distinct host node, the session map **MUST** be encrypted prior to transit across intermediate networks.

5.1. Cipher and Key Derivation Parameters

The cryptographic handoff protocol mandates the following primitives:

- **Key Derivation Function:** Argon2id conforming to [\[RFC9106\]](#). Minimum operational parameters: Memory = 64 MiB (65536 KiB), Iterations = 3, Parallelism = 1.
- **Authenticated Symmetric Encryption:** XChaCha20-Poly1305 AEAD cipher conforming to [\[RFC8439\]](#) with extended 192-bit (24-byte) nonces generated using a cryptographically secure pseudorandom number generator (CSPRNG).
- **Associated Data:** The AEAD associated data **MUST** bind the session UUID and expiration timestamp to prevent ciphertext replay and splicing attacks.

6. Verification and Conformance Audit

Compliance with this specification requires verifiable testing under adversarial boundary conditions:

6.1. The 5-Step Airplane Mode Audit Procedure

1. Initialize the host application or agent runtime with a test corpus containing statutory PII and secrets.
2. Physically disconnect or logically disable all network adapters (Wi-Fi, Ethernet, LTE/5G).
3. Execute complete document sanitization. Verify that output contains strictly surrogate tokens.
4. Simulate a model response containing surrogate tokens and execute local de-tokenization. Confirm 100% string restoration.
5. Inspect host operating system socket tables to confirm exactly zero packets were emitted during the entire test sequence.

7. Conformance Test Vectors

The following test vector illustrates a standardized ZTDS execution sequence:

```
Vector 1.0 (Clinical / Financial Hybrid):
Input Payload:
  "Patient Sarah Connor (MRN: 902-114-88, Phone: +1-555-0144) authorized
  payment using Visa 4111111111111111 to Dr. Marcus Vance."

Sanitized Payload Egress:
  "Patient <tt>[NAME_1]</tt> (MRN: [MRN_1], Phone: [PHONE_1]) authorized
  payment using Visa <tt>[CARD_1]</tt> to Dr. [NAME_2]."
```

```

Volatile Session Map R:
{
  "<tt>[NAME_1]</tt>": "Sarah Connor",
  "<tt>[MRN_1]</tt>": "902-114-88",
  "<tt>[PHONE_1]</tt>": "+1-555-0144",
  "<tt>[CARD_1]</tt>": "4111111111111111",
  "<tt>[NAME_2]</tt>": "Marcus Vance"
}

Inference Response Inbound:
  "Billing confirmed for <tt>[NAME_1]</tt> under clinical file [MRN_1]."
```

```

Restored Client Display:
  "Billing confirmed for Sarah Connor under clinical file 902-114-88."
```

8. IANA Considerations

This document has no IANA actions.

9. Security Considerations

This entire document specifies security and privacy architecture. In conformance with BCP 72 [RFC3552], the following threat scenarios are analyzed:

9.1. Prompt Injection and Training Data Exfiltration

Adversaries executing indirect prompt injection attacks against LLMs attempt to manipulate model context into revealing private user records. Under ZTDS, because raw PII never enters the model context window, prompt injection attacks cannot exfiltrate original credentials; the attacker can at most observe synthetic surrogate labels.

9.2. Local Host Memory Inspection and Swap Analysis

If an adversary achieves root-level compromise of the client operating system, they may inspect volatile memory buffers. To mitigate this, compliant implementations SHOULD use memory locking (e.g., mlock on POSIX systems) to prevent volatile memory from being paged to unencrypted swap disks, and explicitly zero memory buffers upon deallocation.

9.3. Surrogate Token Collision and Ambiguity

If an input prompt naturally contains text matching the surrogate regex syntax (e.g., a software tutorial discussing "[NAME_1]"), the engine **MUST** escape or disambiguate literal brackets using namespace prefixes to prevent accidental de-tokenization collision.

10. Privacy Considerations

In accordance with [RFC6973], ZTDS provides deterministic technical and organizational measures (TOMs) satisfying global privacy statutes:

- **GDPR Article 28 Exemption:** By eliminating cleartext PII transmission across the WAN, downstream foundation model vendors never process personal data in cleartext, preventing SaaS vendor lock-in to burdensome subprocessor compliance chains.
- **Right to be Forgotten (GDPR Article 17):** When AI models or vector databases ingest strictly surrogate tokens, an individual exercising their deletion right requires only erasing the local ephemeral session key; all remote embeddings and logs remain permanently de-identified and mathematically un-linkable.
- **HIPAA Safe Harbor De-Identification:** Replacing all 18 statutory HIPAA identifiers with surrogate tokens fulfills 45 CFR Section 164.514(b) requirements prior to cloud ingestion.

11. References

11.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [RFC8439] Nir, Y. and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols", RFC 8439, DOI 10.17487/RFC8439, June 2018, <<https://www.rfc-editor.org/info/rfc8439>>.
- [RFC9106] Biryukov, A., Dinu, D., Khovratovich, D., and S. Kiviharju, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications", RFC 9106, DOI 10.17487/RFC9106, September 2021, <<https://www.rfc-editor.org/info/rfc9106>>.

11.2. Informative References

- [RFC3552] Rescorla, E. and B. Korver, "Guidelines for Writing RFC Text on Security Considerations", BCP 72, RFC 3552, DOI 10.17487/RFC3552, July 2003, <<https://www.rfc-editor.org/info/rfc3552>>.

- [RFC6973]** Cooper, A., Tschofenig, H., Aboba, B., Peterson, J., Morris, J., Hansen, M., and R. Smith, "Privacy Considerations for Internet Protocols", RFC 6973, DOI 10.17487/RFC6973, July 2013, <<https://www.rfc-editor.org/info/rfc6973>>.
- [ZENODO-ZTDS]** Sibiryakov, I., "Zero-Trust Data Sanitization (ZTDS) Protocol Specification", CERN Zenodo DOI 10.5281/zenodo.22058770, September 2026, <<https://doi.org/10.5281/zenodo.22058770>>.
- [OSF-BENCH]** Sibiryakov, I., "Empirical Latency and Memory Profiling of Client-Side vs Cloud Proxy Data Sanitization", Center for Open Science DOI 10.17605/OSF.IO/5BYJF, September 2026, <<https://doi.org/10.17605/OSF.IO/5BYJF>>.

Author's Address

Ilya Sibiryakov
ZTDS AI Consortium
Tel Aviv
Israel
Email: support@privacyscrubber.com
URI: <https://ztds.ai/>